more computational resources. Serverless platforms provide fully managed auto-scaling features and can scale application instances to zero so that you don’t have to pay anything if there’s nothing to process. You’ll learn more about the serverless model, and you’ll build a serverless application using Spring Native and Spring Cloud Function. Finally, you’ll see how to deploy applications using Knative, a Kubernetes-based serverless platform.

NOTE The source code for the examples in this chapter is available in the Chapter16/16-begin and Chapter16/16-end folders, containing the initial and final states of the project (https://github.com/ThomasVitale/cloud-native-spring-in-action).

16.1 Native images with Spring Native and GraalVM

One of the reasons why Java applications became widely popular was the common platform (the Java Runtime Environment, or JRE), allowing developers to “write them once, run them everywhere,” no matter the operating system. That comes from the way applications are compiled. Rather than compiling the application code directly into machine code (the code understood by operating systems), the Java compiler produces bytecode that a dedicated component (the Java Virtual Machine, or JVM) runs. During execution, the JRE interprets the bytecode into machine code dynamically, allowing the same application executable to run on any machine and OS where a JVM is available. This is called a *just-in-time (JIT) compilation*.

Applications running on the JVM are subject to startup and footprint costs. The startup phase used to be quite long for traditional applications, for which it could even take several minutes. Standard cloud native applications have a much faster startup phase: a few seconds rather than a few minutes. This is